Rješenja — Vježbe 3: Regularni izrazi

> **Verzija s rješenjima — samo za nastavnika.**

V3: Regularni izrazi

I051 Računalno jezikoslovlje - vježbe uz *3. predavanje*.

	Predavanje	Vježbe
1	Osnovni pojmovi	`re.search` i `re.findall`: podudaranje i njegov položaj
2	Gradivni elementi	pretraživanje rječnika hrvatskog korpusa; preciznost i odziv
3	Grupe i zamjena	datumi, iznosi i ponovljene riječi u vijestima; ELIZA na hrvatskom

0. Priprema

Kao u V2: u VSCode-u otvorite mapu s bilježnicom i odaberite Jupyter kernel (gore desno **Select Kernel → Python Environments…**). Paketi su popisani u `requirements.txt`; ćeliju je dovoljno pokrenuti jednom.

Regularni izrazi su u Pythonu ugrađeni (modul `re`), pa ne treba ništa preuzimati. NLTK koristimo samo za hrvatski korpus, kao u V2.

```
%pip install -r requirements.txt
```

U mapi `data` su dvije datoteke korpusa UD Croatian SET (hrvatske vijesti, isti korpus kao u V1 i V2):

- `hr_train.txt` — kao u V2: jedna rečenica po retku, riječi i interpunkcija odvojeni razmakom,
- `hr_vijesti.txt` — **iste rečenice u izvornom obliku**, kako su objavljene: interpunkcija uz riječ, datumi i iznosi onako kako su napisani. Na njoj ćemo tražiti uzorke u tekstu.

1. Prvi koraci: `re.search` i `re.findall`

S predavanja: **uzorak** opisuje stringove koje tražimo, a **podudaranje** je podstring teksta koji odgovara uzorku. U Pythonu za to služi modul `re`:

funkcija	što vraća
`re.search(uzorak, tekst)`	**prvo** podudaranje (objekt `Match`) ili `None`
`re.findall(uzorak, tekst)`	listu **svih** podudaranja

Uzorke uvijek pišemo kao **sirove stringove** `r"…"`, da Python ne bi sam tumačio obrnutu kosu crtu `\`.

1.1 Podudaranje i njegov položaj

```
import re  # modul za regularne izraze, dio Pythona

text = "Grad Zagreb je glavni grad."
```

```
m = re.search(r"Zagreb", text)  # prvo podudaranje
print(m.group())                # podudareni podstring
print(m.span())                 # položaj: (početak, kraj)
print(text[5:11])               # isti podstring izrezan iz teksta
```

Kraj je **prvi znak iza** podudaranja, kao kod rezanja stringova `text[5:11]`. Ako podudaranja nema, `re.search` vraća `None`; zato ga prije upotrebe provjeravamo s `if`:

```
m = re.search(r"zagreb", text)  # malo z: velika i mala slova se razlikuju
if m:
    print("pronađeno:", m.group())
else:
    print("nema podudaranja")

print(re.findall(r"grad", text))  # sva podudaranja: samo malo "grad"
print(re.findall(r"Grad", text))
```

Zašto sirovi string? `\t` u običnom stringu Python pretvara u tabulator, a u sirovom ostaje kako je napisano — dva znaka, `\` i `t`:

```
print("a\tb")    # tabulator
print(r"a\tb")   # obrnuta kosa crta i t
```

1.2 Pretraživanje pravog teksta

Učitajmo vijesti kao jedan dugi string. `len` daje broj **znakova**, a `count("\n")` broj redaka, tj. rečenica.

```
text = open("data/hr_vijesti.txt", encoding="utf-8").read()
print(len(text), "znakova,", text.count("\n"), "rečenica")
print(text[:300])  # početak teksta
```

```
print(len(re.findall(r"Hrvatska", text)))  # koliko puta se javlja »Hrvatska«
print(len(re.findall(r"Hrvatsk", text)))   # i Hrvatske, Hrvatskoj, hrvatski …? (samo s
velikim H)

m = re.search(r"Osijek", text)             # prvo spominjanje Osijeka
text[m.start() - 60 : m.end() + 40]        # malo konteksta oko podudaranja
```

Pokušajte: koliko puta se u vijestima javlja *Zagreb*, a koliko *EU*? Ispišite kontekst prvog spominjanja *Zagreba* (60 znakova lijevo i desno).

```
print(len(re.findall(r"Zagreb", text)), len(re.findall(r"EU", text)))
m = re.search(r"Zagreb", text)
text[m.start() - 60 : m.end() + 60]
```

2. Gradivni elementi

Operatore s predavanja najlakše je isprobati na **rječniku** korpusa: listi svih različnica. Izgradimo ga kao u V2 (hrvatski korpus kao NLTK korpus, samo riječi od slova, malim slovima) i napišimo pomoćnu funkciju `find` koja vraća riječi rječnika u kojima postoji podudaranje s uzorkom.

```
import os, nltk
from nltk.corpus.reader import PlaintextCorpusReader
from nltk.tokenize import WhitespaceTokenizer, LineTokenizer

nltk.data.path.append(os.path.abspath("data"))  # NLTK smije čitati samo dopuštene mape
hr = PlaintextCorpusReader("data", ["hr_train.txt"],
                           word_tokenizer=WhitespaceTokenizer(),
sent_tokenizer=LineTokenizer())
```

```
tokens = list(hr.words())                                          # sve pojavnice
vocab = sorted(set(w.lower() for w in tokens if w.isalpha()))      # rječnik: različnice od
slova
print(len(tokens), "pojavnica,", len(vocab), "različnica")

def find(pattern):
    return [w for w in vocab if re.search(pattern, w)]  # riječi rječnika s podudaranjem
```

2.1 Disjunkcija znakova, raspon i negacija

- `[čć]` — **jedan** znak: č ili ć,
- `[a-z]` — raspon: jedan znak od a do z (poretkom kodnih točaka!),
- `[^aeiou]` — negacija: jedan znak koji **nije** samoglasnik.

Sidra `^` (početak) i `$` (kraj) koristimo da uzorak obuhvati početak ili kraj riječi.

```
print(find(r"^ma[čć]"))          # riječi koje počinju s »mač« ili »mać«
print(len(find(r"[čć]$")))       # koliko riječi završava na č ili ć …
print(find(r"[čć]$")[:8])        # … većinom prezimena
```

Zamka s predavanja: hrvatska slova **nisu** u rasponu `[a-z]`, jer su im kodne točke veće od kodne točke slova z.

```
sentence = "računalno jezikoslovlje proučava hrvatski jezik"
print(re.findall(r"[a-z]+", sentence))       # lomi riječi na č
print(re.findall(r"[a-zčćđšž]+", sentence))  # ispravno
```

```
consonants = find(r"^[^aeiou]+$")  # riječi bez ijednog samoglasnika
print(len(consonants))
print(consonants[:30])             # kratice (bdp, pdv …) i riječi sa slogotvornim r (brz,
prst …)
```

2.2 Ponavljanje i zamjenski znak

operator	značenje
`?`	0 ili 1 put
`*`	0 ili više puta
`+`	1 ili više puta
`{n}`, `{n,m}`	točno n puta, od n do m puta
`.`	bilo koji jedan znak

Zamjenski znak i sidra daju »križaljku«: `^k.ć.$` je riječ od četiri slova kojoj je prvo slovo k, a treće ć.

```
print(find(r"^k.ć.$"))          # križaljka
print(find(r"^hrvatsk"))        # sve riječi koje počinju s »hrvatsk«
print(find(r"^hrvatsk[aeiou]m?$"))  # nastavak: jedan samoglasnik i možda m
print(len(find(r"^.{17,}$")), find(r"^.{17,}$")[:5])  # riječi od barem 17 slova
```

Pokušajte: pronađite riječi koje završavaju na *-ost* (budućnost, djelatnost …). Koliko ih ima? Ima li među njima nešto što nije imenica na -ost?

```
words = find(r"ost$")
```

```
print(len(words))
print(words[:20])   # i engleska riječ »cost« te pogreška »budućnost«
```

2.3 Disjunkcija | i grupiranje

`|` bira između **cijelih izraza**, a zagrade određuju dokle disjunkcija seže: `^(pod|nad|pred)` je početak riječi *pod*, *nad* ili *pred*.

```
print(len(find(r"^(pod|nad|pred)")), find(r"^(pod|nad|pred)")[:10])
print(len(find(r"(ski|ška|sko)$")))   # pridjevi na -ski, -ška, -sko
```

Zadatak 1

Napišite izraze i ispišite broj pronađenih riječi i prvih deset:

1. riječi koje završavaju na *-ica* (učiteljica, ulica …),
2. riječi od **točno tri slova** koje počinju s *p*,
3. superlativi: riječi koje počinju s *naj* i završavaju na *iji* (najnoviji …).

Što od pronađenog nije ono što ste htjeli (lažno pozitivni)?

```
patterns = {
    "-ica": r"ica$",
    "p + 2 slova": r"^p..$",
    "naj…iji": r"^naj.*iji$",
}
for name, pattern in patterns.items():
    words = find(pattern)
    print(f"{name:12} {len(words):4}  {words[:10]}")
# lažno pozitivni: npr. america, apocalyptica (strane riječi na -ica); kratice pbz, pci,
pdv
```

2.4 Preciznost i odziv

Ponovimo primjer s predavanja na cijelom korpusu: tražimo riječ *je*. Korpus je već rastavljen na pojavnice, pa znamo **točan** odgovor: to su pojavnice *je* i *Je*. Za svaki izraz brojimo:

- **preciznost** = točno pronađene / sve pronađene (kvare je lažno pozitivni),
- **odziv** = točno pronađene / sve koje treba naći (kvare ga lažno negativni).

```
def precision_recall(pattern, tokens, correct):
    found = [t for t in tokens if re.search(pattern, t)]  # pojavnice s podudaranjem
    tp = sum(1 for t in found if t in correct)            # točno pronađene
    total = sum(1 for t in tokens if t in correct)        # sve koje treba naći
    return tp / len(found), tp / total

for pattern in [r"je", r"[jJ]e", r"\b[jJ]e\b"]:
    p, r = precision_recall(pattern, tokens, {"je", "Je"})
    print(f"{pattern:12} preciznost {p:.3f}   odziv {r:.3f}")
```

Bez sidara izraz pronalazi i *jezik*, *lijep*, *bolje* … (niska preciznost). Velika slova malo popravljaju odziv, jer rečenica rijetko počinje s *Je*.

Zadatak 2

U vijestima (`text`) potražite **godine**: četveroznamenkaste brojeve koji počinju s 19 ili 20.

1. Koliko podudaranja daje naivni izraz `r"[0-9]{4}"`, a koliko izraz za godine? (Za godine upotrijebite

`\b` i disjunkciju `|`.)

2. Ispišite deset najčešćih četveroznamenkastih brojeva **koji nisu godine** — to su lažno pozitivni naivnog izraza. Što su?

```
from collections import Counter

naive = re.findall(r"[0-9]{4}", text)
years = re.findall(r"\b19\d\d\b|\b20\d\d\b", text)
print(len(naive), len(years))

not_years = [n for n in naive if not re.search(r"^(19|20)", n)]
Counter(not_years).most_common(10)
# 1244 = Rezolucija UN-a o Kosovu; 1000, 5000 = iznosi
# ali 1865, 1842, 1861 su godine iz 19. stoljeća: lažno negativni našeg izraza za godine!
```

3. Grupe i zamjena

3.1 Kratice za klase znakova

kratica	isto što i	značenje
`\d`	`[0-9]`	znamenka
`\w`	slova, znamenke, `_`	riječni znak (u Pythonu 3 i č, ć, đ, š, ž)
`\s`	razmak, `\t`, `\n`	praznina

Velika slova su negacije: `\D`, `\W`, `\S`.

```
print(re.findall(r"\w+", "Računalno jezikoslovlje, 2027."))  # \w poznaje i hrvatska slova
print(re.findall(r"\d+", "Rezolucija 1244 iz 1999., 3 točke"))  # nizovi znamenki
```

3.2 Grupe za hvatanje: datumi u vijestima

Datum u vijestima često ima oblik *15. listopada 2009.* Zagrade u uzorku su **grupe za hvatanje**: pamte svoj dio podudaranja. Kad uzorak ima grupe, `re.findall` vraća **n-torke** sadržaja grupa.

```
date = r"\b(\d{1,2})\. ([a-zčćđšž]+) (\d{4})\."   # dan. mjesec godina.
dates = re.findall(date, text)
print(len(dates))
dates[:5]
```

```
from collections import Counter

Counter(month for day, month, year in dates).most_common(5)  # najčešći mjeseci
```

U **zamjeni** `re.sub(uzorak, zamjena, tekst)` sadržaj grupa dohvaćamo s `\1`, `\2`, `\3`. Preslagivanjem grupa datum zapisujemo drugim redoslijedom:

```
example = "Sporazum je potpisan 22. lipnja 2007. u Bruxellesu."
print(re.sub(date, r"\3., \1. \2", example))      # godina na početak
print(re.sub(r"\d", "#", "OIB 12345678901"))      # zamjena bez grupa: skrivanje znamenki
```

3.3 Povratna referenca: ponovljene riječi

`\1` **unutar uzorka** podudara se s istim tekstom koji je uhvatila grupa 1. Tako nalazimo ponovljene riječi — česte tipfelere:

```
repeated = r"\b(\w+) \1\b"
for m in re.finditer(repeated, text):           # finditer: sva podudaranja kao objekti
Match
    print(text[m.start() - 30 : m.end() + 15].replace("\n", " "))
```

`re.finditer` je kao `re.findall`, ali daje cijele objekte `Match`, pa imamo i položaj (`start`, `end`) za ispis konteksta.

Za razmisliti: koji su od pronađenih pravi tipfeleri, a koji lažno pozitivni? (Pogledajte *NATO-u u*: zašto `\b` vidi granicu riječi iza crtice?)

3.4 Grupa bez hvatanja i pogled unaprijed

- `(?: … )` grupira, ali **ne pamti** — `re.findall` tada vraća cijelo podudaranje,
- `(?= … )` je **pogled unaprijed**: uvjet što mora slijediti, bez trošenja znakova.

Iznosi u eurima: broj (možda s točkama i decimalnim zarezom) iza kojeg slijedi riječ *eura* — ali bez nje u rezultatu.

```
print(re.findall(r"(ha)+", "hahaha"), re.findall(r"(?:ha)+", "hahaha"))  # razlika u
findall

amounts = re.findall(r"\d+(?:[.,]\d+)*(?= eura)", text)
print(len(amounts))
amounts[:10]
```

Pokušajte: pronađite iznose u **dolarima**. Što ako između broja i valute stoji *milijuna* (npr. *4,5 milijuna dolara*)? Proširite izraz.

```
print(re.findall(r"\d+(?:[.,]\d+)*(?= dolara)", text)[:10])
re.findall(r"\d+(?:[.,]\d+)*(?= (?:milijuna |milijardi )?dolara)", text)[:15]
```

3.5 ELIZA na hrvatskom

S predavanja: ELIZA je **kaskada zamjena**. Unos pretvori u velika slova, zamijeni zamjenice (*JA SAM → TI SI*), a zatim provjerava pravila redom; prvo koje odgovara daje odgovor.

`re.match` je kao `re.search`, ali traži podudaranje samo **na početku** stringa; zato pravila počinju s `.*`.

```
def eliza(t):
    t = t.upper()                                   # 1. velika slova
    t = re.sub(r"\bMOJ\b", "TVOJ", t)               # 2. zamjenice
    t = re.sub(r"\bJA SAM\b", "TI SI", t)
    if re.match(r".*\bTI SI (TUŽAN|TUŽNA)\b", t):   # 3. pravila redom
        return re.sub(r".*\bTI SI (TUŽAN|TUŽNA)\b.*", r"ŽAO MI JE ŠTO SI \1.", t)
    if re.match(r".*\bUVIJEK\b", t):
        return "MOŽEŠ LI SE SJETITI PRIMJERA?"
    return "RECI MI NEŠTO VIŠE."                    # 4. ništa ne odgovara

for line in ["Ja sam tužna većinu vremena.", "Uvijek nas zbog nečega gnjave.", "Trebam
pomoć."]:
    print(f"{line:35} {eliza(line)}")
```

Zadatak 3

Proširite ELIZU s dva pravila iz razgovora na predavanju (dodajte ih **prije** općeg odgovora):

1. *Trebam X* → *ŠTO BI TI ZNAČILO DA DOBIJEŠ X?* (X je jedna riječ; uhvatite je grupom),
2. ako se spominje netko iz obitelji (*majka*, *otac*, *brat*, *sestra* …) → *RECI MI VIŠE O SVOJOJ OBITELJI.*

Isprobajte na rečenicama ispod. Zašto pravilo za obitelj ne prepoznaje *s majkom*, i kako biste to popravili?

```
def eliza2(t):
    t = t.upper()
    t = re.sub(r"\bMOJ\b", "TVOJ", t)
    t = re.sub(r"\bJA SAM\b", "TI SI", t)
    if re.match(r".*\bTI SI (TUŽAN|TUŽNA)\b", t):
        return re.sub(r".*\bTI SI (TUŽAN|TUŽNA)\b.*", r"ŽAO MI JE ŠTO SI \1.", t)
    if re.match(r".*\bUVIJEK\b", t):
        return "MOŽEŠ LI SE SJETITI PRIMJERA?"
    if re.match(r".*\bTREBAM (\w+)", t):
        return re.sub(r".*\bTREBAM (\w+).*", r"ŠTO BI TI ZNAČILO DA DOBIJEŠ \1?", t)
    if re.match(r".*\b(MAJKA|OTAC|BRAT|SESTRA)\b", t):
        return "RECI MI VIŠE O SVOJOJ OBITELJI."
    return "RECI MI NEŠTO VIŠE."

for line in ["Trebam pomoć, toliko je sigurno.", "Majka se brine o meni.",
             "Možda bih se naučio slagati s majkom.", "Danas je lijep dan."]:
    print(f"{line:40} {eliza2(line)}")
# »s majkom«: pravilo traži samo oblik MAJKA. Popravak: korijeni s nastavcima, npr.
r".*\b(MAJK|OTA?C|BRAT|SESTR)\w*\b"
```

Za razmisliti: engleska ELIZA radi s nekoliko desetaka pravila. Koliko bi ih trebalo za hrvatski, gdje svaka imenica ima sedam padeža, a pridjevi se slažu u rodu (*tužan*, *tužna*)? To je **morfologija** s predavanja P2 — i jedan od razloga zašto su pravila ustupila mjesto modelima naučenima iz podataka.